



Министерство науки и высшего образования Российской Федерации
Калужский филиал федерального государственного автономного
образовательного учреждения высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИУК Информатика и управление

КАФЕДРА ИУКЗ Системы автоматического управления

ОТЧЕТ
по НАУЧНО-ИССЛЕДОВАТЕЛЬСКОЙ РАБОТЕ
НА ТЕМУ:

Исследование метода автоматического построения
математической модели системы автоматического
управления по иерархической структурной схеме

Студент группы ИУКЗ-71Б

Исаев А.А.

(Подпись)

(И.О. Фамилия)

Руководитель НИР

Макаренков А.М.

(Подпись)

(И.О. Фамилия)

Оценка руководителя НИР

Баллов, 30-50

(Дата)

Оценка за защиту

Баллов, 30-50

(Дата)

Итоговая оценка

Баллов

(по пятибалльной шкале)

Комиссия: _____

Калуга, 2026

**Министерство науки и высшего образования Российской Федерации
Калужский филиал федерального государственного автономного образовательного
учреждения высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)**

УТВЕРЖДАЮ
Заведующий кафедрой ИУКЗ
_____ (Д.В. Мельников)
«___» _____ 2026 г.

З А Д А Н И Е

**на научно-исследовательскую работу
на тему:**

Исследование метода автоматического построения математической модели
системы автоматического управления по иерархической структурной схеме

Задание:

1. Проанализировать способы перехода от структурной схемы к математической модели и существующие средства моделирования систем автоматического управления.
2. Формализовать структурную схему как граф и описать метод построения модели в пространстве состояний через матрицу соединений.
3. Описать раскрытие иерархических схем и решение алгебраических контуров с оценкой их обусловленности.
4. Реализовать метод в вычислительном ядре с консольными сценариями исследования.
5. Провести вычислительные эксперименты: корректность на эталонных схемах, алгебраические контуры, иерархия, затраты времени, интегрирование собранной модели; обобщить результаты.

Дата выдачи задания «___» _____ 2026 г.

Руководитель

Макаренков А.М.

(подпись, дата)

(И.О. Фамилия)

Студент

Исаев А.А.

(подпись, дата)

(И.О. Фамилия)

Содержание

Перечень сокращений и обозначений.....	4
Введение.....	5
1. Сбор и анализ материала, анализ литературы.....	7
1.1. Актуальность темы исследования.....	7
1.2. Способы перехода от схемы к модели.....	7
1.3. Объект, предмет, цель и задачи исследования.....	8
2. Проведение научного исследования, расчётов.....	9
2.1. Модели типовых звеньев.....	9
2.2. Запись схемы таблицей соединений.....	10
2.3. Порядок решения уравнений.....	11
2.4. Контурные без инерционных звеньев.....	12
2.5. Схемы с подсистемами.....	12
2.6. Реализация в вычислительном ядре.....	13
2.7. Эксперимент 1: проверка на схемах с известным ответом.....	14
2.8. Эксперимент 2: контурные без инерционных звеньев.....	15
2.9. Эксперимент 3: схемы с подсистемами.....	16
2.10. Эксперимент 4: время сборки модели.....	17
2.11. Эксперимент 5: решение полученных уравнений.....	18
3. Обобщение полученных результатов.....	20
3.1. Научные и практические результаты.....	20
3.2. Ограничения и направления дальнейшей работы.....	20
Заключение.....	21
Список использованных источников.....	22
Приложение А. Листинги программы.....	23

Перечень сокращений и обозначений

САУ – система автоматического управления

ПФ – передаточная функция $W(s)$

ПИД – пропорционально-интегрально-дифференциальный регулятор

A, B, C, D – матрицы модели $\frac{dx}{dt} = Ax + Bu, y = Cx + Du$

M, N – таблицы соединений: какой вход блока к какому выходу или внешнему воздействию подключён

κ – число обусловленности контура без инерционных звеньев

ε – машинная точность, $\approx 2,2 \cdot 10^{-16}$

RK4 – метод Рунге–Кутты 4-го порядка с постоянным шагом

RK45 – метод Дорманда–Принса с автоматическим выбором шага

h – шаг интегрирования

API – программный интерфейс сервера

Введение

Научно-исследовательская работа выполнялась по направлению подготовки 27.03.04 «Управление в технических системах». Она является исследовательской частью выпускной квалификационной работы «Веб-ориентированная система визуального моделирования систем автоматического управления».

Систему автоматического управления (САУ) обычно изображают структурной схемой. На ней каждое звено — прямоугольник со своей передаточной функцией, а стрелки показывают, куда идёт сигнал. Чтобы рассчитать, как система поведёт себя во времени, схему нужно превратить в систему уравнений.

Вручную это делают по правилам структурных преобразований. Последовательные звенья перемножают, параллельные складывают, а контур с обратной связью сворачивают по формуле $W/(1+W)$. Для простой схемы это быстро. Для схемы с несколькими контурами и подсистемами преобразование становится долгим, и в нём легко ошибиться. Программа моделирования должна делать этот шаг сама и без ошибок.

В работе исследуется метод, при котором программа строит уравнения всей схемы сразу по списку соединений. Ей не нужно искать, в каком порядке сворачивать контуры. Метод реализован в вычислительном ядре разрабатываемой системы.

Цель работы — исследовать метод автоматического построения математической модели САУ по структурной схеме с подсистемами и проверить, насколько он точен, надёжен и быстр.

Для этого решались задачи:

- сравнить способы перехода от структурной схемы к уравнениям;
- описать, как схема записывается в виде таблицы соединений и как по ней собирается модель;
- описать, как обрабатываются подсистемы и контуры без инерционных звеньев;

- реализовать метод в вычислительном ядре и написать консольные сценарии проверки;
- проверить метод на схемах с заранее известным ответом, оценить надёжность расчёта и затраты времени;
- сравнить численные методы, которыми решаются полученные уравнения.

1. Сбор и анализ материала, анализ литературы

1.1. Актуальность темы исследования

Структурная схема — основной язык теории автоматического управления [2, 3]. Человеку удобно рассуждать звеньями и связями. Компьютеру удобно решать систему дифференциальных уравнений. Перевод одного в другое — главная задача любой программы блочного моделирования.

Самый известный инструмент такого моделирования, MATLAB/Simulink, сейчас недоступен российским вузам. Отечественные SimInTech и Engее — коммерческие программы с закрытым кодом. В них пользователь не видит, какие уравнения программа составила по его схеме. Для учебного процесса полезна открытая среда, которая показывает полученную модель и объясняет, почему схему нельзя рассчитать. Поэтому исследование метода построения модели и его надёжности актуально.

1.2. Способы перехода от схемы к модели

Можно выделить три способа. Их сравнение приведено в таблице 1.

Таблица 1 – Способы перехода от структурной схемы к модели

Способ	Как работает	Ограничения
Правила структурных преобразований	Схема вручную сворачивается до одной передаточной функции	Трудоёмко при нескольких контурах, возможны ошибки
Поблочный расчёт	На каждом шаге времени блоки считаются по очереди, по ходу сигнала	Не работает, если сигнал обходит контур мгновенно; не даёт общих уравнений системы
Сборка по таблице соединений (исследуемый метод)	Уравнения всех звеньев объединяются, связи подставляются, система решается целиком	Только линейные звенья

Поблочный расчёт использовался в первой версии вычислительного ядра (весна 2026 г.). Он прост, но у него два недостатка. Во-первых, он не может рассчитать схему, в которой сигнал проходит по контуру без задержки. Во-вторых, он не даёт общих уравнений системы, а именно по ним

определяют устойчивость. Поэтому ядро было переведено на исследуемый метод.

1.3. Объект, предмет, цель и задачи исследования

Объект исследования — линейные непрерывные САУ, заданные структурными схемами с подсистемами.

Предмет исследования — метод автоматического построения модели по таблице соединений схемы: его точность, поведение в контурах без инерционных звеньев, работа с подсистемами и затраты времени.

Цель и задачи исследования сформулированы во введении.

2. Проведение научного исследования, расчётов

2.1. Модели типовых звеньев

Каждое звено описывается уравнениями в форме «вход – состояние – выход». Состояние — это величины, которые звено «запоминает» между моментами времени, например накопленное значение интегратора. Уравнения звена номер i имеют вид

$$\frac{dx_i}{dt} = A_i x_i + B_i u_i, y_i = C_i x_i + D_i u_i, \quad (1)$$

где u_i — вход звена, y_i — выход, x_i — состояние. Коэффициенты A_i, B_i, C_i, D_i определяются параметрами звена. Например, у усилителя состояния нет и $y = ku$. У звена $\frac{K}{Ts+1}$ одно состояние: $\frac{dx}{dt} = -\frac{x}{T} + \frac{u}{T}$, $y = Kx$. Поддерживаемые звенья перечислены в таблице 2.

Таблица 2 – Типовые звенья и их модели

Звено	Передающая функция	Состояни й	Мгновенная передача
Усилитель	k	0	да
Сумматор	$\pm u_1 \pm u_2 \pm \dots$	0	да
Интегратор	$\frac{k}{s}$	1	нет
Звено 1-го порядка	$\frac{K}{Ts+1}$	1	нет
Колебательное звено	$\frac{K\omega_0^2}{s^2 + 2\zeta\omega_0 s + \omega_0^2}$	2	нет
Передающая функция	$\frac{N(s)}{D(s)}$	степень D(s)	если степени N и D равны
ФНЧ Баттерворта	порядок n , частота среза ω_c	n	нет
ПИД-регулятор	$K_p + \frac{K_i}{s} + \frac{K_d N s}{s+N}$	2	да

Последний столбец показывает, передаёт ли звено изменение входа на выход сразу, без задержки ($D_i \neq 0$). Это важно для раздела 2.4: если контур состоит только из таких звеньев, сигнал проходит его мгновенно, и такой контур приходится решать отдельно.

2.2. Запись схемы таблицей соединений

Схему удобно рассматривать как граф: блоки — это точки, а связи — стрелки от выхода одного блока ко входу другого. Каждый вход получает ровно один сигнал: либо выход другого блока, либо внешнее воздействие r (ступенчатый вход).

Сначала уравнения всех звеньев записываются вместе, одним общим набором. Входы всех блоков собираются в один вектор u_b , выходы — в вектор y_b , состояния — в вектор x . Затем связи схемы записываются таблицей из нулей и единиц:

$$u_b = M y_b + N r. \quad (2)$$

Здесь M — таблица соединений: на пересечении строки «вход блока» и столбца «выход блока» стоит единица, если они соединены. Таблица N так же описывает подключение внешних воздействий. Подставив соединения в уравнения звеньев, получаем уравнения для выходов блоков:

$$(I - D_d M) y_b = C_d x + D_d N r, \quad (3)$$

где C_d и D_d — коэффициенты всех звеньев, записанные вместе, а I — единичная матрица. Если эти уравнения имеют единственное решение, выходы блоков выражаются через состояния и воздействия: $y_b = Y_x x + Y_r r$. После подстановки получается модель всей схемы:

$$\frac{dx}{dt} = A x + B r, y = C x + D r, \quad (4)$$

$$A = A_d + B_d M Y_x, B = B_d (M Y_r + N), C = S_y Y_x, D = S_y Y_r + S_r, \quad (5)$$

где S_y и S_r указывают, какие сигналы подключены к осциллографам. Главное свойство метода: ему не нужно знать, где на схеме контуры и в каком порядке их сворачивать. Любая обратная связь — это просто ещё одна единица в таблице M .

Рассмотрим схему с обратной связью на рисунке 1. Объект $\frac{1}{s-1}$ сам по себе неустойчив, его охватывает усилитель $K=3$ и отрицательная обратная связь.

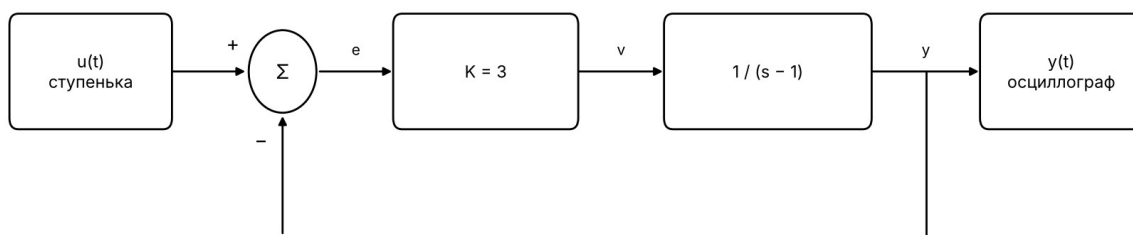


Рисунок 1 – Схема «Стабилизация неустойчивого объекта»

У этой схемы три выхода блоков (сумматор, усилитель, объект) и четыре входа. Таблица соединений M приведена в таблице 3. Например, единица в строке «усилитель» и столбце «выход сумматора» означает, что выход сумматора подключён ко входу усилителя. Вход «+» сумматора подключён к ступенчатому воздействию, поэтому он записан не в M , а в N .

Таблица 3 – Таблица соединений M для схемы рисунка 1

Вход блока	Выход сумматора	Выход усилителя	Выход объекта
сумматор, вход «+» (от ступеньки)	0	0	0
сумматор, вход «-»	0	0	1
усилитель	1	0	0
объект $\frac{1}{s-1}$	0	1	0

Программа получает $A=-2$, $B=3$, $C=1$, $D=0$, то есть $\frac{dx}{dt} = -2x + 3r$, $y=x$.

Это совпадает с ручным расчётом по формуле замкнутого контура:

$$W = \frac{3}{s-1+3} = \frac{3}{s+2}.$$

Полос системы равен -2 . Он отрицательный, значит, замкнутая система устойчива, хотя сам объект неустойчив.

2.3. Порядок решения уравнений

Решать все уравнения для выходов сразу — лишняя работа. В большинстве схем сигнал идёт от блока к блоку без петель, и выходы можно найти по очереди, подставляя уже найденные. Поэтому программа сначала находит участки схемы, где сигнал замыкается в петлю мгновенно, без инерционных звеньев. Остальные блоки она считает простой подстановкой

по ходу сигнала. Систему уравнений приходится решать только на найденных участках (для поиска таких участков используется алгоритм Тарьяна [8]).

2.4. Контур без инерционных звеньев

Такой контур называют алгебраическим: сигнал обходит его мгновенно, поэтому его нельзя рассчитать шаг за шагом во времени — нужно решить уравнение. Простейший пример — усилитель с положительной обратной связью:

$$y = k(u + y) \Rightarrow y = \frac{k}{1 - k} u. \quad (6)$$

При $k=1$ уравнение не имеет решения. При k , близком к единице, решение есть, но оно очень чувствительно: небольшая неточность в k сильно меняет результат. Например, при $k=0,999$ получаем $y=999u$, а при $k=0,998$ — уже $y=499u$.

Насколько контур чувствителен, программа оценивает числом обусловленности κ . Оно показывает, во сколько раз относительная погрешность коэффициентов может вырасти в результате. Для одного усилителя в петле

$$\kappa = \frac{1 + |k|}{|1 - k|}, \quad (7)$$

а для контура из нескольких блоков используется общая формула через сингулярные числа матрицы контура [4]. Компьютер хранит числа примерно с 16 верными знаками. Если κ больше $1/\sqrt{\varepsilon} \approx 6,7 \cdot 10^7$, в ответе остаётся меньше половины верных знаков. В этом случае программа не выдаёт сомнительный результат, а сообщает, какие блоки образуют контур, и предлагает изменить коэффициенты или добавить инерционное звено.

2.5. Схемы с подсистемами

Подсистема — это блок, внутри которого находится своя схема со своими входами и выходами. Перед сборкой модели программа

«раскрывает» подсистемы: переносит их блоки на верхний уровень и соединяет связи, проходившие через границу подсистемы, напрямую. Чтобы имена не совпадали, к имени каждого блока добавляется путь подсистемы, например `measurement::sensor_path::filter`. Если выход подсистемы соединён прямо с её входом, сигнал просто проходит насквозь. Две копии одной подсистемы получают разные пути и не влияют друг на друга.

Пример подсистемы показан на рисунке 2. Блок «Измерительный тракт» сам является подсистемой второго уровня. Модель строится по всей схеме сразу, а программа запоминает, из какой подсистемы пришла каждая переменная состояния (таблица 4).

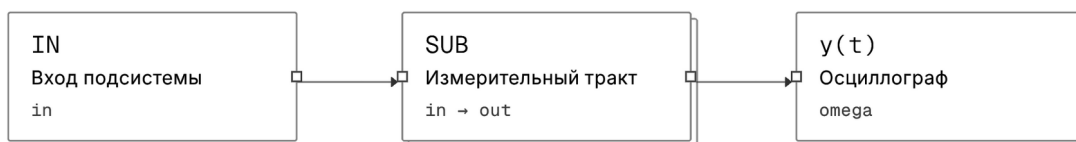


Рисунок 2 – Подсистема «Измерение» с вложенной подсистемой

Таблица 4 – Переменные состояния многоуровневой схемы и их подсистемы

х	Блок	Тип звена	Подсистема
x1	actuator_lag	звено 1-го порядка	plant / actuator
x2	oscillator	колебательное	plant
x3	oscillator	колебательное	plant

2.6. Реализация в вычислительном ядре

Метод реализован на Python с библиотеками NumPy и SciPy [5, 6]. Модули ядра перечислены в таблице 5.

Таблица 5 – Модули вычислительного ядра

Модуль	Назначение
app/core/block_specs.py	описание звеньев, их входов, выходов и допустимых параметров
app/validation	проверка схемы: типы блоков, соединения, один источник на каждый вход
app/simulation/hierarchy.py	раскрытие подсистем
app/simulation/assembly.py	уравнения звеньев, таблица соединений, контуры без инерционных звеньев, итоговая модель
app/simulation/solvers.py	численные методы: Эйлер, RK4, RK45, точное решение
scripts/nir_assembly_study.py	консольная проверка метода (разделы 2.7–2.10)
scripts/nir_solver_study.py	консольное сравнение численных методов (раздел 2.11)

Все таблицы и графики этой главы получаются одной командой:

```
cd backend
python scripts/nir_assembly_study.py
python scripts/nir_solver_study.py
```

Функция сборки модели приведена в листинге 1 приложения А, решение контуров без инерционных звеньев — в листинге 2.

2.7. Эксперимент 1: проверка на схемах с известным ответом

Для проверки подготовлено 16 схем: последовательное и параллельное соединение, разветвление сигнала, отрицательная и положительная обратная связь, контур с ПИД-регулятором, контуры без инерционных звеньев, подсистемы, две копии одной подсистемы, подсистема с двумя входами и выходами, фильтр Баттерворта. Для каждой схемы передаточная функция $G(s)$ рассчитана вручную по правилам структурных преобразований.

Передаточную функцию собранной модели можно вычислить по её коэффициентам: $C(sI - A)^{-1}B + D$. Она сравнивалась с ручным расчётом в четырёх точках s . Ошибка считалась как $\frac{\max|C(sI - A)^{-1}B + D - G(s)|}{\max(1, |G(s)|)}$.

Результаты приведены в таблице 6; последний столбец — результат поблочного расчёта первой версии ядра.

Таблица 6 – Сравнение собранных моделей с ручным расчётом

Схема	n	Входы × выходы	Контур без инерции	Ошибка метода	Поблочный расчёт
series	2	1 × 1	0	1.8e-16	1.8e-16
parallel	2	1 × 1	0	1.6e-16	1.6e-16
branching	1	1 × 2	0	0.0e+00	0.0e+00
negative_feedback	2	1 × 1	0	4.7e-16	4.7e-16
positive_feedback	1	1 × 1	0	0.0e+00	0.0e+00
pid_loop	3	1 × 1	0	6.1e-16	6.1e-16
biproper_loop	1	1 × 1	1	1.1e-16	не собирает
static_loop	0	1 × 1	1	0.0e+00	не собирает
subsystem_feedback	1	1 × 1	0	1.1e-16	1.1e-16
flat_subsystem_feedback	1	1 × 1	0	1.1e-16	1.1e-16
two_instances	2	1 × 1	0	5.6e-17	5.6e-17
two_instances_with_inner_scopes	2	1 × 3	0	5.6e-17	5.6e-17
pass_through	1	1 × 1	0	0.0e+00	0.0e+00
nested_feedback	1	1 × 1	0	1.1e-16	1.1e-16

mimo_subsystem	1	2×2	0	0.0e+00	0.0e+00
butterworth	4	1×1	0	8.9e-16	8.9e-16

Метод собрал все 16 схем. Наибольшая ошибка — 8.9e-16: это уровень погрешности округления чисел в компьютере, то есть результат совпадает с ручным расчётом. Поблочный расчёт не справился с 2 схемами — обе содержат контур без инерционных звеньев.

2.8. Эксперимент 2: контуры без инерционных звеньев

Исследовался контур $y = k(u + y)$ при приближении k к единице. Для каждого k результат программы сравнивался с точным $y = \frac{k}{1-k}$. Кроме того, k изменялся на одну миллиардную долю, и измерялось, во сколько раз это изменение выросло в результате. Данные приведены в таблице 7 и на рисунке 3.

Таблица 7 – Решение контура $y = k(u + y)$

k	к (формула)	к (программа)	у программы	у точное	Ошибка	Рост погрешности	Итог
0,5	3	7,12	1	1	0.0e+00	2	решён
0,9	19	39	9	9	0.0e+00	10	решён
0,99	199	399	99	99	0.0e+00	100	решён
0,999	2e+03	4e+03	999	999	0.0e+00	1e+03	решён
0,999999	2e+06	4e+06	999999	999999	0.0e+00	1e+06	решён
0,999999999	2e+12	—	—	1e+12	—	—	отказ: плохо обусловлен
1	∞	—	—	∞	—	—	отказ: вырожден
1,5	5	11,4	-3	-3	1.5e-16	2	решён
3	2	6,83	-1,5	-1,5	1.5e-16	0,5	решён

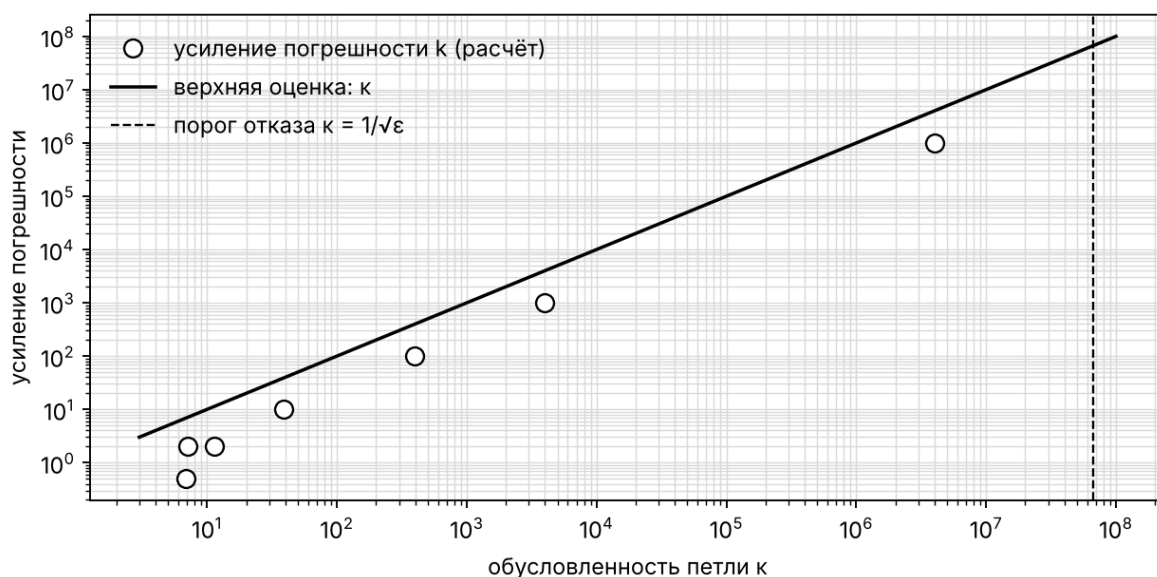


Рисунок 3 – Рост погрешности в зависимости от обусловленности контура

Пока k невелико, программа решает контур точно. Чем ближе k к единице, тем больше k и тем сильнее растёт погрешность. Все точки на рисунке лежат ниже прямой k , значит, оценка надёжна. При $k=1-10^{-12}$ ($k \approx 2 \cdot 10^{12}$) программа отказывается от расчёта, а при $k=1$ сообщает, что контур не имеет решения. Значение k из программы примерно вдвое больше формулы для одного усилителя, потому что в схеме контур проходит через два блока — сумматор и усилитель.

2.9. Эксперимент 3: схемы с подсистемами

Сначала сравнивались две записи одной и той же системы: контур, проходящий через подсистему, и та же схема без подсистем. Выходы совпали полностью (расхождение $0.0e+00$). Затем звено 1-го порядка в контуре с обратной связью вкладывалось в подсистемы глубиной от 1 до 32 уровней (таблица 8).

Таблица 8 – Раскрытие вложенных подсистем

Глубина	Блоков после раскрытия	n	Раскрытие, мс	Сборка, мс	Отличие $y(t)$
1	4	1	0,08	0,43	0.0e+00
2	4	1	0,10	0,49	0.0e+00
4	4	1	0,26	0,57	0.0e+00
8	4	1	0,18	0,65	0.0e+00
16	4	1	0,26	1,00	0.0e+00
32	4	1	0,64	1,61	0.0e+00

При любой глубине после раскрытия остаются те же четыре блока, модель первого порядка и тот же выход. Время раскрытия растёт пропорционально глубине и даже при 32 уровнях составляет доли миллисекунды.

2.10. Эксперимент 4: время сборки модели

Время сборки измерялось на цепочке звеньев 1-го порядка длиной от 2 до 256 звеньев. Рассматривались цепочка без обратных связей и цепочка, где каждое звено охвачено своей обратной связью. Каждый замер повторялся 15 раз, в таблицу 9 и на рисунок 4 занесено среднее (медиана).

Таблица 9 – Время сборки модели

п	Блоков	Сборка, мс	Блоков (с контурами)	Сборка, мс (с контурами)
2	4	0,29	6	0,41
8	10	0,55	18	1,10
32	34	1,81	66	3,34
64	66	3,15	130	6,73
128	130	6,70	258	19,21
256	258	20,21	514	57,66

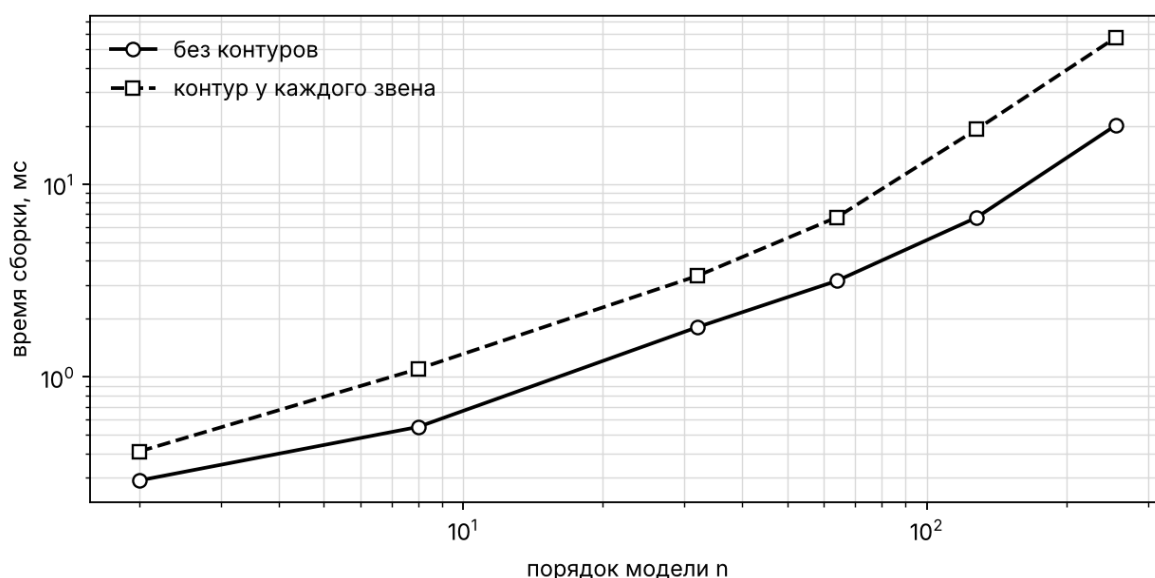


Рисунок 4 – Время сборки модели в зависимости от её порядка

Даже схема из более чем 500 блоков с 256 контурами собирается быстрее чем за десятую долю секунды. Учебные схемы из десятков блоков

собираются за доли миллисекунды. Поэтому модель можно пересобирать при каждом движении ползунка параметра, и пользователь видит результат сразу.

2.11. Эксперимент 5: решение полученных уравнений

Собранная модель $\frac{dx}{dt} = Ax + Br$ решается численно: время делится на шаги длиной h , и состояние пересчитывается от шага к шагу. Сравнивались три метода. Метод Эйлера на каждом шаге делает один расчёт производной. Метод Рунге–Кутты 4-го порядка (RK4) делает четыре расчёта и усредняет их, поэтому точнее. Метод RK45 сам подбирает шаг под заданную точность. Эталоном служило точное решение, которое можно получить именно потому, что метод дал матрицы A и B .

Порядок метода показывает, как быстро уменьшается ошибка при уменьшении шага. Он вычислялся как $p = \log_2 \frac{\varepsilon(h)}{\varepsilon(h/2)}$, где $\varepsilon(h)$ — наибольшая ошибка при шаге h . Результаты для контура с ПИД-регулятором приведены в таблице 10.

Таблица 10 – Ошибка методов на контуре с ПИД-регулятором

h, с	Эйлер	RK4	RK45	p Эйлер	p RK4
0,2	1.20e+02	3.89e-03	7.35e-09	—	—
0,1	2.11e+00	2.37e-04	7.35e-09	5,83	4,04
0,05	2.75e-01	1.46e-05	7.35e-09	2,94	4,02
0,025	1.06e-01	9.08e-07	7.37e-09	1,38	4,01
0,0125	4.74e-02	5.65e-08	7.43e-09	1,15	4,01

Порядок метода Эйлера приближается к 1, метода RK4 — к 4, как и предсказывает теория [7]. На практике это значит, что при уменьшении шага вдвое ошибка метода Эйлера уменьшается вдвое, а ошибка RK4 — в 16 раз. Чтобы ошибка не превышала 0,001, методу Эйлера нужно в 128–512 раз больше шагов, чем RK4. Поэтому в системе используются RK4 и RK45, а метод Эйлера оставлен для сравнения.

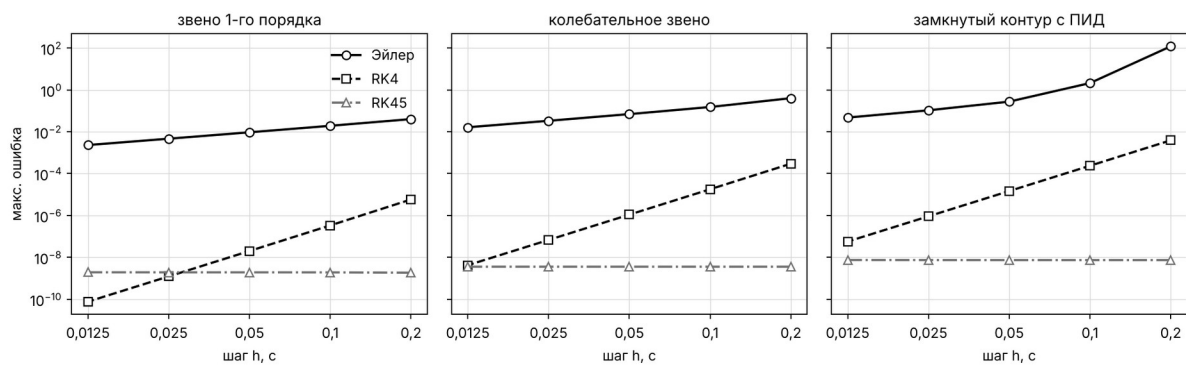


Рисунок 5 – Ошибка численных методов в зависимости от шага

3. Обобщение полученных результатов

3.1. Научные и практические результаты

В ходе НИР получены следующие результаты.

- Описан метод, который строит модель всей схемы по таблице соединений и не требует ручного сворачивания контуров.
- Описаны раскрытие подсистем и решение контуров без инерционных звеньев с проверкой надёжности результата.
- На 16 схемах с известным ответом ошибка метода не превысила 9e-16. Поблочный расчёт не справился с 2 из них.
- Показано, что число обусловленности надёжно оценивает рост погрешности, и обоснован порог, после которого программа отказывается от расчёта.
- Подтверждено, что подсистемы не меняют модель, а время их раскрытия растёт пропорционально глубине.
- Модель из более чем 500 блоков собирается за десятки миллисекунд.
- Экспериментально подтверждены порядки точности численных методов.

Практический результат — вычислительное ядро веб-системы визуального моделирования. Оно автоматически строит модель любой линейной схемы и показывает пользователю, из каких блоков получилась каждая часть модели.

3.2. Ограничения и направления дальнейшей работы

- метод рассчитан на линейные звенья; для нелинейных элементов (насыщение, реле) нужна другая форма модели;
- входные воздействия пока только ступенчатые;
- на основе собранной модели в курсовом проекте разрабатывается модуль анализа запасов устойчивости.

Заключение

В результате НИР исследован метод автоматического построения математической модели системы автоматического управления по структурной схеме с подсистемами.

Метод записывает соединения схемы таблицей, объединяет уравнения всех звеньев и получает модель всей системы. Подсистемы предварительно раскрываются, а контуры без инерционных звеньев решаются с проверкой надёжности результата.

Эксперименты показали, что метод совпадает с ручным расчётом по правилам структурных преобразований с точностью до погрешности округления. Он рассчитывает схемы, недоступные поблочному расчёту, корректно отказывается от ненадёжных контуров, не зависит от глубины вложенности подсистем и работает быстро даже для моделей сотен порядков.

Метод реализован в вычислительном ядре веб-системы визуального моделирования и является основой выпускной квалификационной работы.

Список использованных источников

1. Финошин А.В., Корнюшин Ю.П., Краснощеченко В.И. Научно-исследовательская работа: практикум. – Калуга: КФ МГТУ им. Н.Э. Баумана, 2024. – 17 с.
2. Бесекерский В.А., Попов Е.П. Теория систем автоматического регулирования. – М.: Директмедиа, 2021. – 747 с.
3. Ким Д.П. Теория автоматического управления. Т. 1. Линейные системы. – М.: Физматлит, 2016. – 312 с.
4. Голуб Дж., Ван Лоун Ч. Матричные вычисления. – М.: Мир, 1999. – 548 с.
5. SciPy API Reference [Электронный ресурс]. – URL: <https://docs.scipy.org/doc/scipy/reference/> (дата обращения: 01.10.2026).
6. NumPy Documentation [Электронный ресурс]. – URL: <https://numpy.org/doc/> (дата обращения: 01.10.2026).
7. Hairer E., Nørsett S.P., Wanner G. Solving Ordinary Differential Equations I: Nonstiff Problems. – Berlin; Heidelberg: Springer, 2008. – 528 p.
8. Кормен Т., Лейзерсон Ч., Ривест Р., Штайн К. Алгоритмы: построение и анализ. 3-е изд. – М.: Вильямс, 2013. – 1328 с.

Приложение А. Листинги программы

Листинг 1 – Сборка модели схемы (app/simulation/assembly.py)

```
def assemble_linear_model(flat: Diagram) -> LinearModel:
    """Assemble a validated, flattened diagram into (A, B, C, D)."""

    sources: list[StepSource] = []
    scopes: list[ScopeOutput] = []
    source_index: dict[str, int] = {}
    scope_index: dict[str, int] = {}

    dynamic: list[tuple[Block, BlockRealization]] = []
    output_index: dict[str, int] = {} # block id -> row of y_b
    input_offset: dict[str, int] = {} # block id -> first row of u_b
    states: list[StateEntry] = []
    total_inputs = 0

    for block in flat.blocks:
        if block.type in SOURCE_TYPES:
            source_index[block.id] = len(sources)
            sources.append(
                StepSource(
                    block.id,
                    get_numeric_parameter(block.parameters, "amplitude", 1.0),
                    get_numeric_parameter(block.parameters, "t0", 0.0),
                )
            )
            continue
        if block.type in SINK_TYPES:
            label = str(block.parameters.get("label") or "").strip() or block.id
            reference = (
                get_numeric_parameter(block.parameters, "reference", 0.0)
                if "reference" in block.parameters
                else None
            )
            scope_index[block.id] = len(scopes)
            scopes.append(ScopeOutput(block.id, label, reference))
            continue

        realization = block_realization(block)
        if realization.b.shape[1] != len(block.input_ports):
            raise ModelAssemblyError(
                [f"Блок '{block.id}': число входов не совпадает с моделью звена."]
            )
        output_index[block.id] = len(dynamic)
        input_offset[block.id] = total_inputs
        total_inputs += len(block.input_ports)
        for local in range(realization.order):
            label = block.id if realization.order == 1 else f"{block.id}.x{local + 1}"
            states.append(StateEntry(block.id, block.type, local, label))
        dynamic.append((block, realization))

    realizations = [realization for _, realization in dynamic]
    n = sum(r.order for r in realizations)
    n_out = len(dynamic)
    n_src = len(sources)
    p = len(scopes)

    a_d = _block_diagonal([r.a for r in realizations], n, n)
    b_d = _block_diagonal([r.b for r in realizations], n, total_inputs)
    c_d = _block_diagonal([r.c for r in realizations], n_out, n)
    d_d = _block_diagonal([r.d for r in realizations], n_out, total_inputs)
    x0 = np.concatenate([r.x0 for r in realizations]) if realizations else np.zeros(0)

    m = np.zeros((total_inputs, n_out))
    nr = np.zeros((total_inputs, n_src))
    s_y = np.zeros((p, n_out))
    s_r = np.zeros((p, n_src))
```

```

blocks_by_id = {block.id: block for block in flat.blocks}

for connection in flat.connections:
    target = blocks_by_id[connection.to_block]
    if target.type in SINK_TYPES:
        row_matrix_y, row_matrix_r, row = s_y, s_r, scope_index[target.id]
    else:
        row = input_offset[target.id] +
target.input_ports.index(connection.to_port)
        row_matrix_y, row_matrix_r = m, nr
        if connection.from_block in source_index:
            row_matrix_r[row, source_index[connection.from_block]] = 1.0
        else:
            row_matrix_y[row, output_index[connection.from_block]] = 1.0

g = d_d @ m
owners = [block.id for block, _ in dynamic]
y_solution, loops = _solve_algebraic_equations(g, np.hstack([c_d, d_d @ nr]),
owners)
y_x, y_r = y_solution[:, :n], y_solution[:, n:]

return LinearModel(
    a=a_d + b_d @ m @ y_x,
    b=b_d @ (m @ y_r + nr),
    c=s_y @ y_x,
    d=s_y @ y_r + s_r,
    x0=x0,
    states=states,
    sources=sources,
    scopes=scopes,
    algebraic_loops=loops,
)

```

Листинг 2 – Решение контуров без инерционных звеньев

(app/simulation/assembly.py)

```

def _solve_algebraic_equations(
    g: np.ndarray,
    rhs: np.ndarray,
    owners: list[str],
) -> tuple[np.ndarray, list[AlgebraicLoop]]:
    """Solve (I - g) Y = rhs component by component."""

    solution = np.zeros_like(rhs)
    loops: list[AlgebraicLoop] = []
    eps = np.finfo(float).eps
    for component in _topological_components(g):
        index = np.asarray(component)
        local_rhs = rhs[index] + g[index] @ solution # unsolved rows of solution are
still zero
        if index.size == 1 and g[index[0], index[0]] == 0.0:
            solution[index] = local_rhs
            continue

        loop_blocks = sorted({owners[i] for i in component})
        loop_gain = g[np.ix_(index, index)]
        matrix = np.eye(index.size) - loop_gain
        smallest = float(np.linalg.svd(matrix, compute_uv=False)[-1])
        # Relative sensitivity of the loop solution to perturbations of the
        # loop coefficients: kappa = (1 + ||G||) / sigma_min(I - G). For a
        # scalar loop y = k y + ... this is (1 + |k|) / |1 - k|.
        scale = 1.0 + float(np.linalg.norm(loop_gain, 2))
        if smallest <= eps * scale * index.size:
            raise ModelAssemblyError(
                [
                    "Алгебраическая петля не имеет решения (матрица I - D·M
вырождена): "
                    + " → ".join(loop_blocks)
                ]
            )
    return solution, loops

```

```

        + ". Измените коэффициенты петли или добавьте динамическое звено."
    ]
)
condition = scale / smallest
if condition > MAX_LOOP_CONDITION:
    raise ModelAssemblyError(
        [
            f"Алгебраическая петля плохо обусловлена (κ = {condition:.3g}): "
            + " → ".join(loop_blocks)
            + ". Результат расчёта был бы недостоверным."
        ]
    )
    solution[index] = lu_solve(lu_factor(matrix), local_rhs)
    loops.append(AlgebraicLoop(blocks=loop_blocks, condition_number=condition))
return solution, loops

```

Листинг 3 – Эксперимент с контуром $y = k(u + y)$

(scripts/nir_assembly_study.py)

```

def algebraic_loops() -> tuple[list[dict], list[tuple[float, float, float]]]:
    """Положительная обратная связь без динамики:  $y = k \cdot (u + y)$ , точное решение  $y = k/(1 - k)$ ."""
    rows, curve = [], []
    for k in (0.5, 0.9, 0.99, 0.999, 0.999999, 1.0 - 1e-12, 1.0, 1.5, 3.0):
        case = static_loop_case(k, ("+", "+"))
        exact = k / (1.0 - k) if k != 1.0 else float("inf")
        formula = (1.0 + abs(k)) / abs(1.0 - k) if k != 1.0 else float("inf")
        row = {"k": f"{k:.12g}", "κ скалярной петли": f"{formula:.3g}"}
        try:
            model = compile_model(Diagram.model_validate(case.diagram)).model
            value = float(model.d[0, 0])
            kappa = model.algebraic_loops[0].condition_number
            error = abs(value - exact) / max(1.0, abs(exact))
            # Чувствительность: насколько относительная погрешность к усиливается в
            решении петли.
            delta = 1e-9
            shifted = compile_model(Diagram.model_validate(static_loop_case(k * (1 +
            delta), ("+", "+")).diagram)).model
            amplification = abs(float(shifted.d[0, 0]) - value) / abs(value) / delta
            row.update({"κ (программа)": f"{kappa:.3g}", "y программы":
            f"{value:.10g}",
                        "y точное": f"{exact:.10g}", "отн. ошибка": f"{error:.1e}",
                        "усиление погрешности k": f"{amplification:.3g}", "итог":
            "решён"})
            curve.append((kappa, amplification, k))
        except DiagramCompilationError as exc:
            reason = "вырожден" if "не имеет решения" in exc.errors[0] else "плохо
            обусловлен"
            row.update({"κ (программа)": "-", "y программы": "-", "y точное":
            f"{exact:.4g}",
                        "отн. ошибка": "-", "усиление погрешности k": "-", "итог":
            f"отказ: {reason}")
            rows.append(row)
    return rows, curve

```

Листинг 4 – Метод Эйлера (app/simulation/solvers.py)

```

def euler_integrate(
    rhs: Rhs,
    x0: np.ndarray,
    t_grid: np.ndarray,
    *,
    breakpoints: Sequence[float] = (),
) -> np.ndarray:
    """Explicit Euler on the given grid:  $x_{k+1} = x_k + h \cdot f(t_k, x_k)$ .

```

First-order reference method for the numerical study; it is not offered by the API. The derivative is taken at the left end of each step, so a jump at a breakpoint grid node takes effect from the step that starts there.

```

"""
_validate_time_grid(t_grid)
del breakpoints # the left-point rule never evaluates past the step start
if x0.size == 0:
    return np.zeros((0, t_grid.size))

trajectory = np.zeros((x0.size, t_grid.size))
trajectory[:, 0] = x0
x = x0.astype(float).copy()
for index in range(t_grid.size - 1):
    left, right = float(t_grid[index]), float(t_grid[index + 1])
    with np.errstate(over="ignore", invalid="ignore"): # divergence is reported
below
        x = x + (right - left) * rhs(left, x)
        if not np.all(np.isfinite(x)):
            raise SolverError(f"Эйлер: состояние перестало быть конечным в момент
t={right:g}.")
        trajectory[:, index + 1] = x
    return trajectory

```